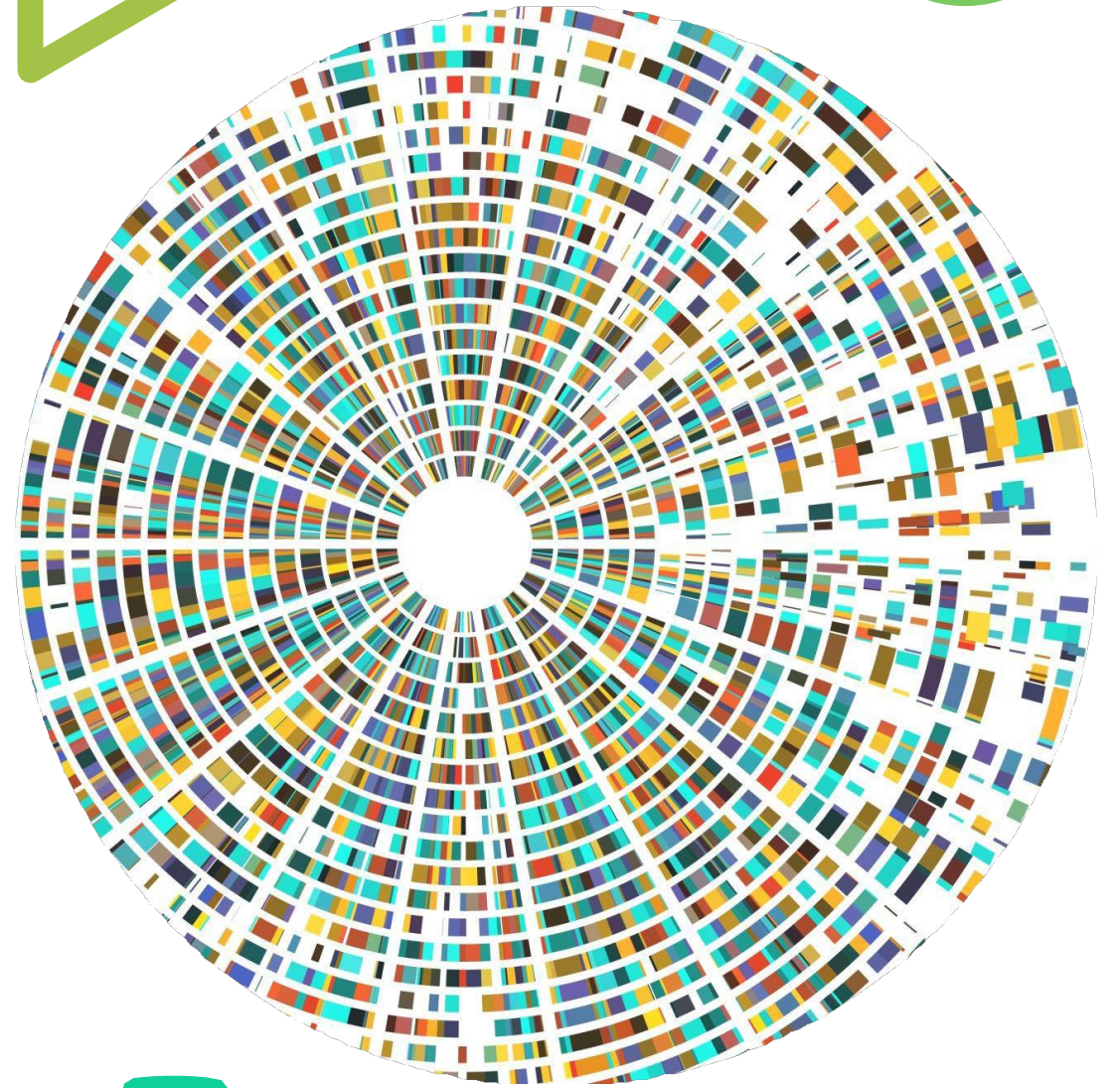




UT-3

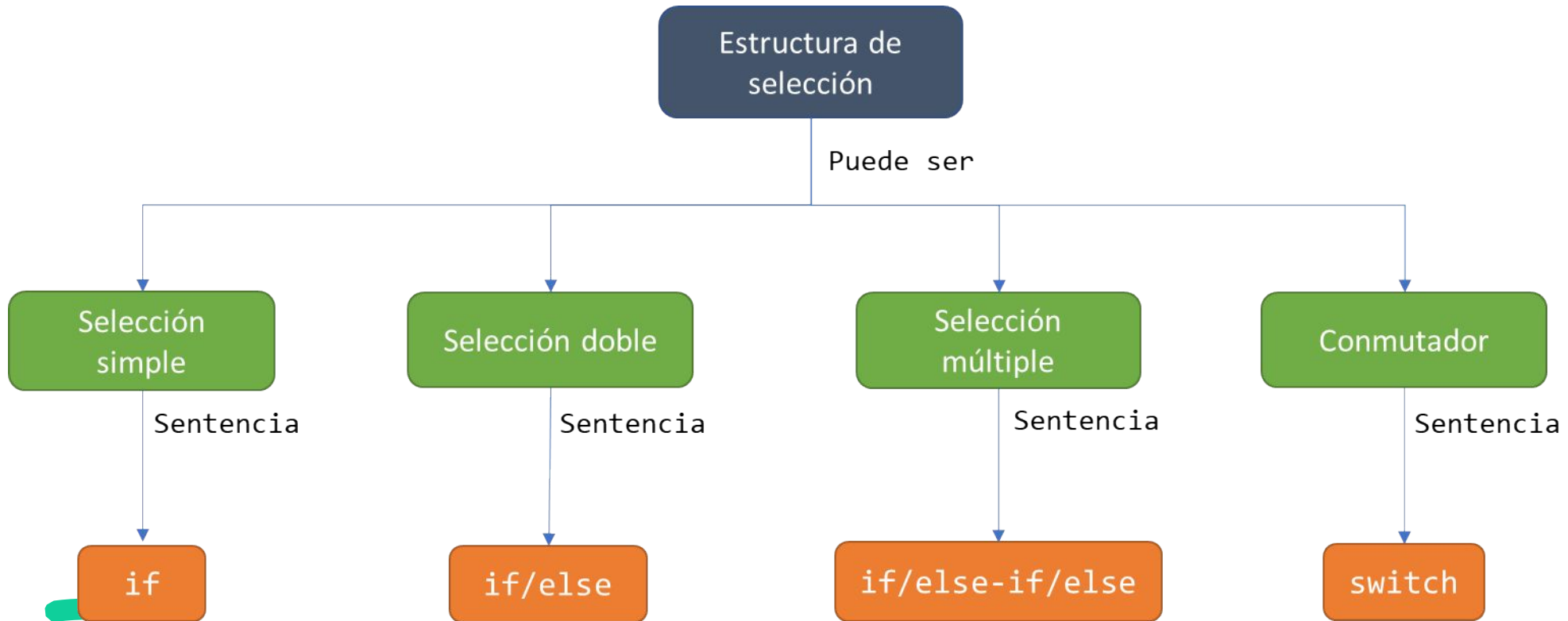
Estructuras de Control

1º DAM/DAW
(2024-2025)



1 - Estructuras de Selección

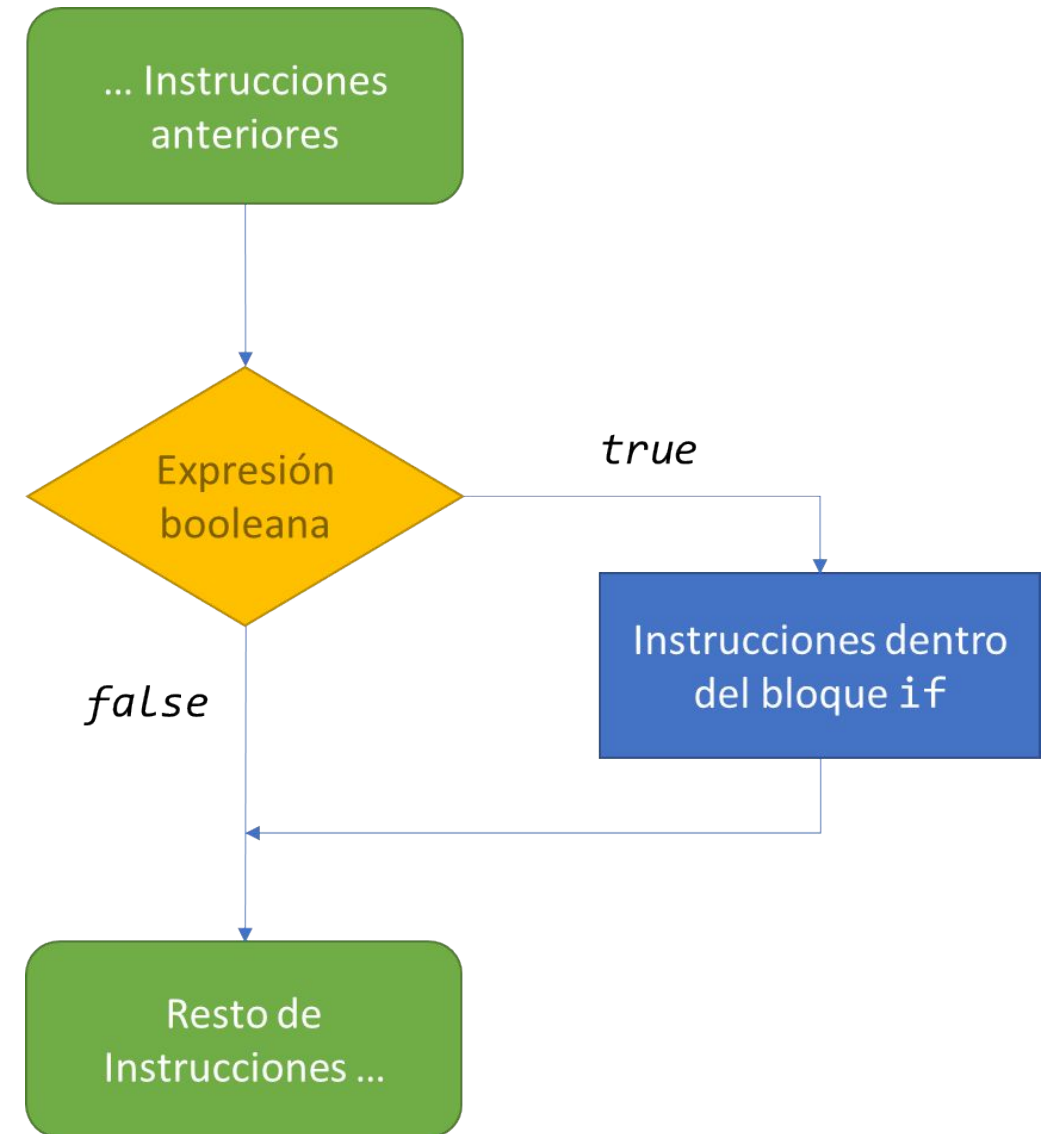
Las **estructuras de selección** permiten tomar decisiones sobre qué conjunto de instrucciones a ejecutar en un punto del programa. O sea, seleccionar qué código se ejecuta en un momento determinado entre caminos alternativos.



1.1 - Selección simple

La estructura de **selección simple** permite controlar el hecho de que se ejecute un conjunto de instrucciones si y sólo si se cumple la condición lógica (es decir, el resultado de evaluar la condición lógica es igual a `true`). En caso contrario, no se ejecutan.

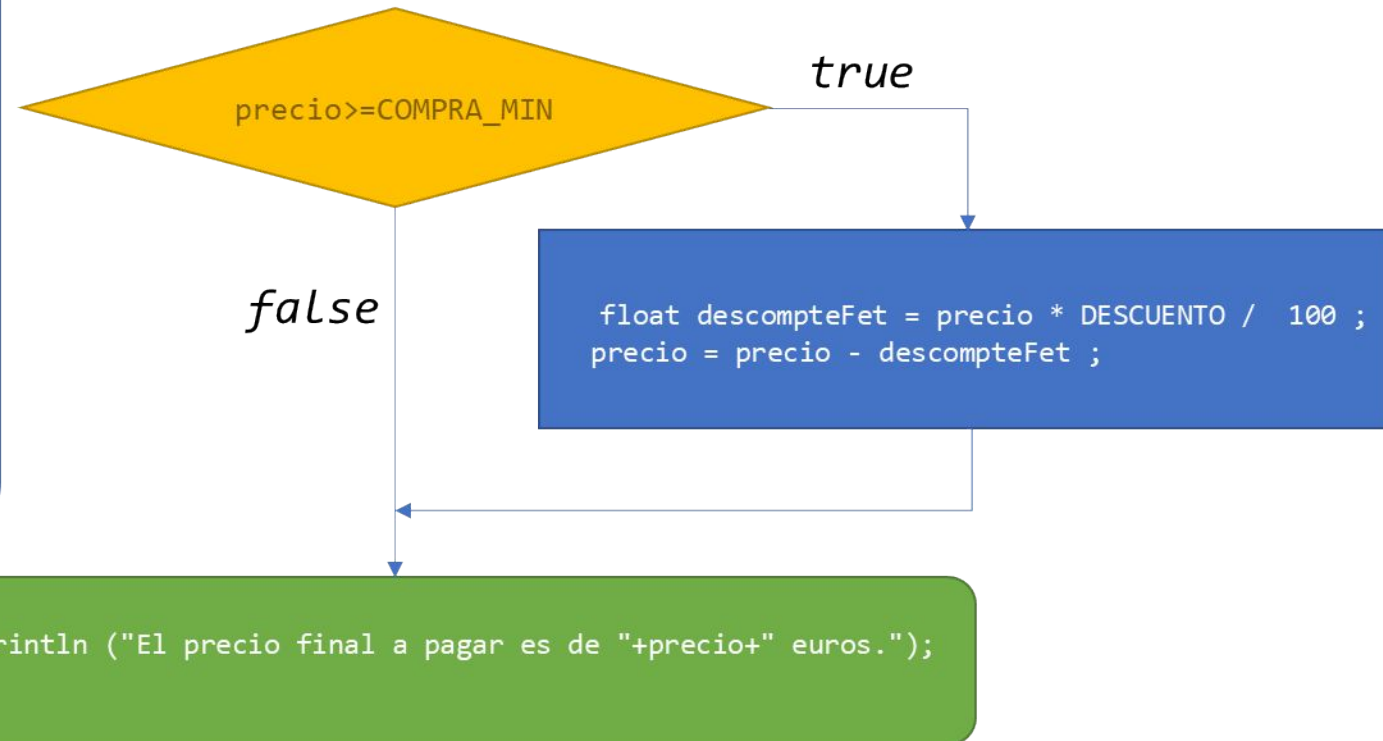
```
Instrucciones anteriores ...  
if (expresión booleana) {  
    sentencia1; sentencia2;  
    .  
    .  
    sentenciaN;  
}  
Resto de instrucciones ...
```



1.1 - Selección simple (ejemplo)

```
...  
System.out.println ("¿Cuál es el precio del producto, en euros?");  
float precio = lector.nextFloat ( ) ;
```

```
1 import java.util.Scanner ;  
2 // Un programa que calcula descuentos.  
3 public class Descuento {  
4     // PASO 1 Se establece el descuento y el precio mínimo  
5     // Se hace un descuento del 8%.  
6     public static final float DESCUENTO = 8;  
7     // Se hace descuento por compras de un mínimo de 50 euros.  
8     public static final float COMPRA_MIN = 50;  
9     public static void main (String [ ] args) {  
10        // PASO 2 Se solicita el precio  
11        System.out.println ( "¿Cuál es el precio del producto, en euros?" );  
12        // PASO 3 Se lee el precio  
13        var lector = new Scanner (System.in);  
14        float precio = lector.nextFloat ( ) ;  
15        lector.nextLine ( ) ;  
16        // PASO 4 Estructura de selección simple.  
17        // Si la expresión evalúa true, ejecuta el bloque dentro del if.  
18        // En caso contrario se ignora.  
19        if (precio>=COMPRA_MIN){  
20            float descuent = precio * DESCUENTO / 100 ;  
21            precio = precio - descuent ;  
22        }  
23        // PASO 5 Salida de resultado por consola  
24        System . out . println ( "El precio final a pagar es de " + precio + " euros." ) ;  
25    }  
26 }
```

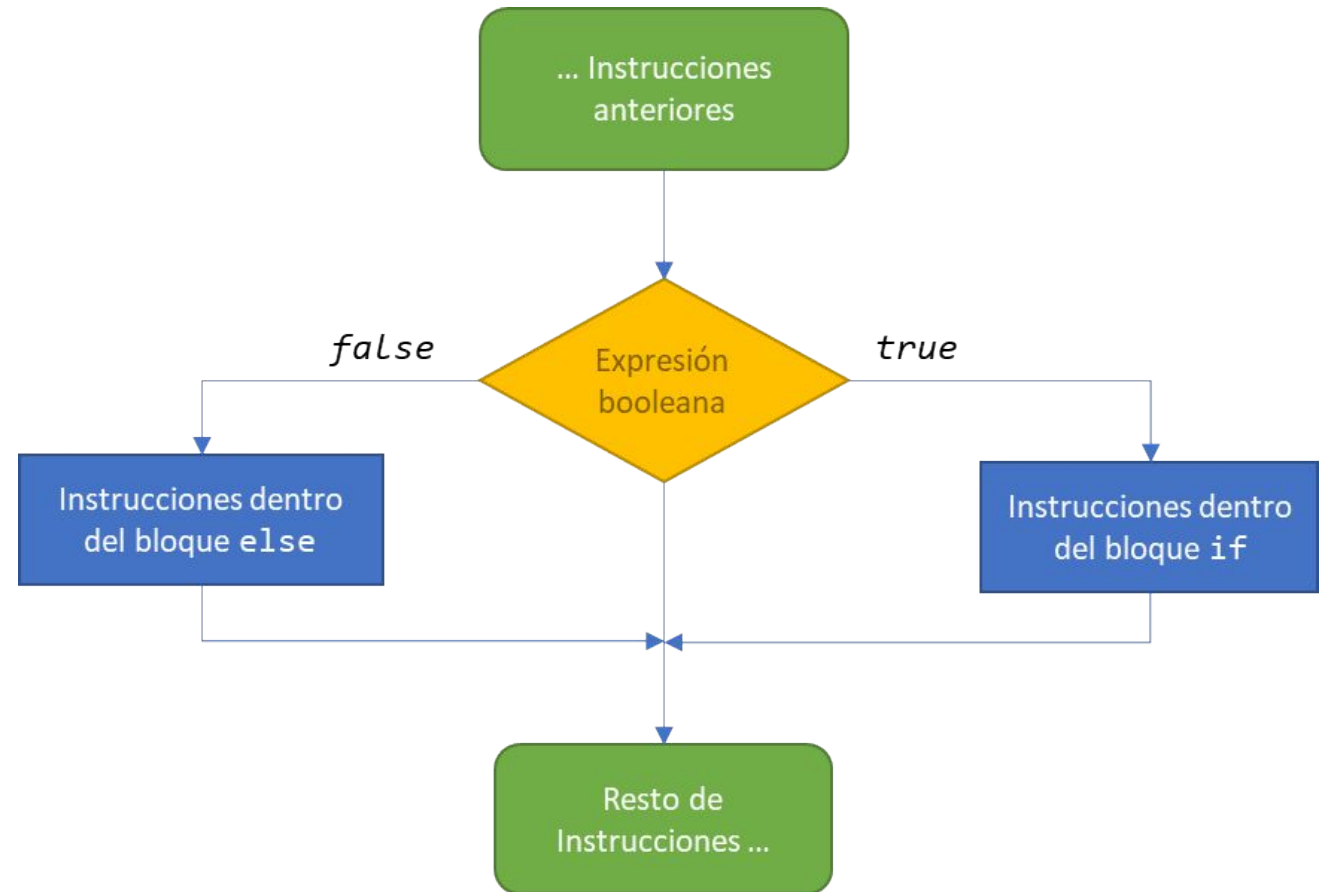


1.2 - Selección Doble

La estructura de **selección doble** permite controlar el hecho de que se ejecute un conjunto de instrucciones, sólo si se cumple la condición lógica, y que se ejecute otro, sólo si no se cumple la condición lógica.

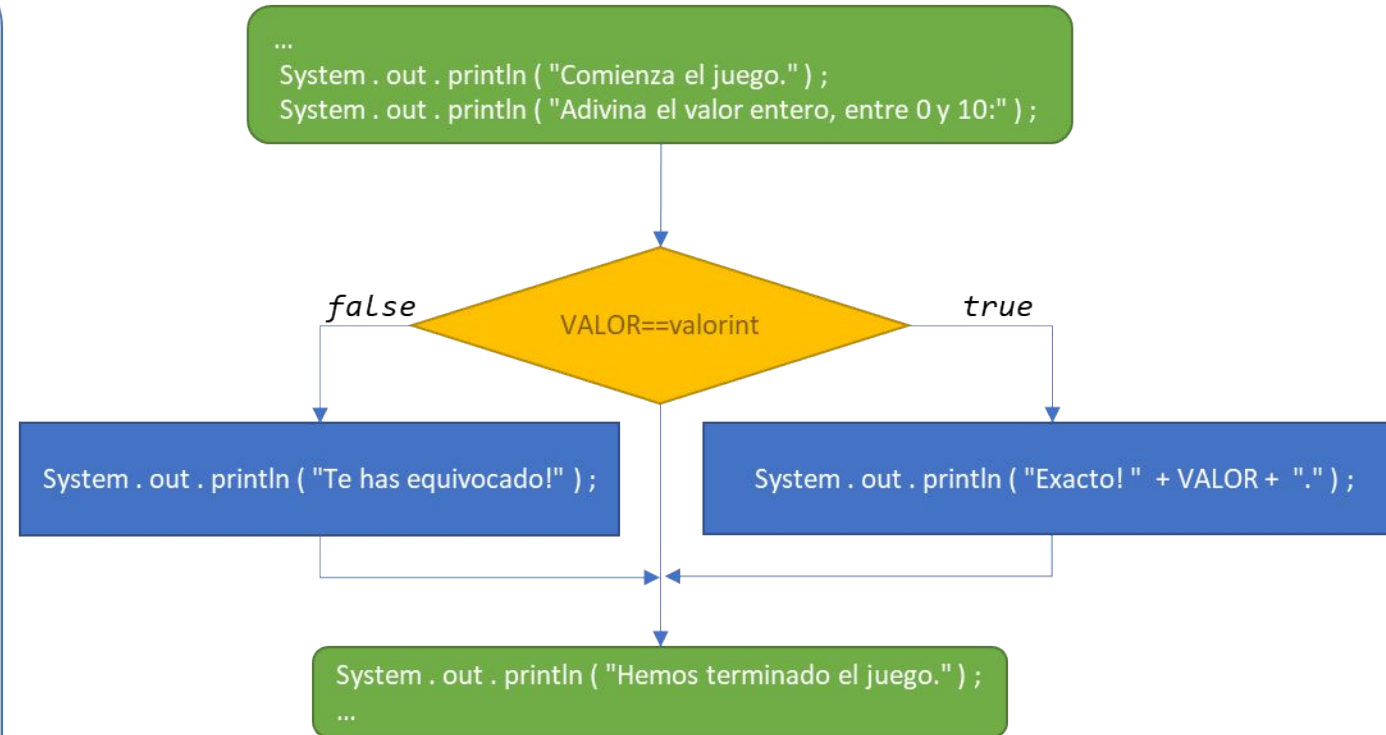
... Instrucciones anteriores

```
if (expresión booleana) {  
    Instrucciones la expresión booleana es true;  
    Bloque if;  
}  
else {  
    Instrucciones si la expresión booleana es false;  
    Bloque else;  
}  
Resto de instrucciones ...
```



1.2 - Selección Doble (Ejemplo)

```
1 import java.util.Scanner ;
2 // Un programa en el que hay que adivinar un número.
3 public class Adivina {
4     // PASO 1 El valor a adivinar será el 4.
5     public static final int VALOR = 4 ;
6     public static void main ( String [ ] args ) {
7         // PASO 2 Solicitar el número
8         System . out . println ( "Comienza el juego." ) ;
9         System . out . println ( "Adivina el valor entero, entre 0 y 10:" ) ;
10        // PASO 3 Leer el número.
11        Scanner lector = new Scanner ( System.in ) ;
12        int valorint = lector.nextInt();
13        lector.nextLine ( ) ;
14        // PASO 4
15        // Estructura de selección doble.
16        // O adivina o se falla.
17        if ( VALOR== valorint ) {
18            // PASO 4.1
19            // Si la expresión evalúa true, ejecuta el bloque dentro del if.
20            System . out . println ( "Exacto! Era" + VALOR + "." ) ;
21        } else {
22            // PASO 4.2
23            // Si la expresión evalúa false, ejecuta el bloque dentro del else.
24            System . out . println ( "Te has equivocado!" ) ;
25        }
26        System . out . println ( "Hemos terminado el juego." ) ;
27    }
28 }
```

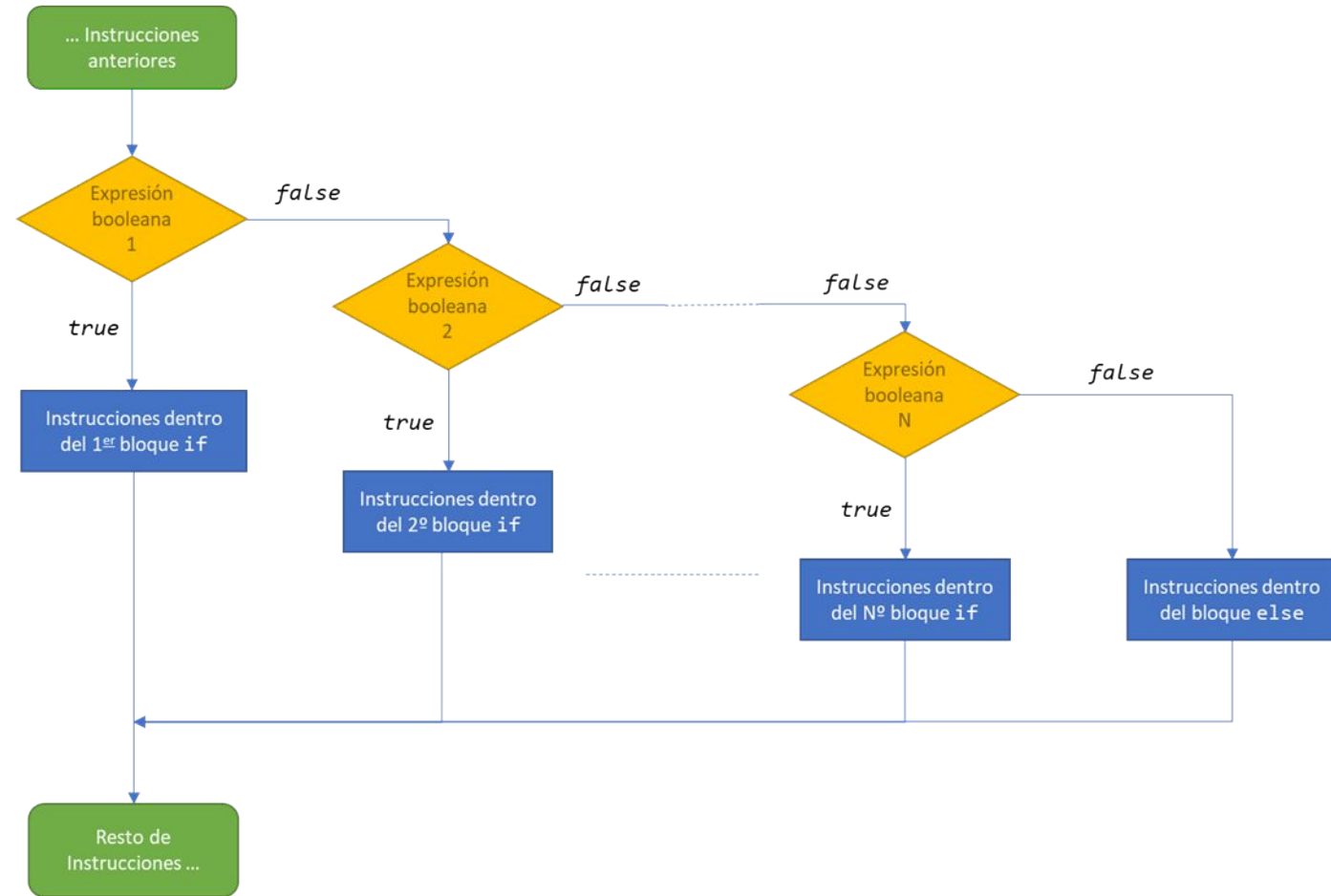


1.3 - Selección múltiple

La estructura de **selección múltiple** permite controlar el hecho de que al cumplirse un caso entre un conjunto finito de casos se ejecute el conjunto de instrucciones correspondiente.

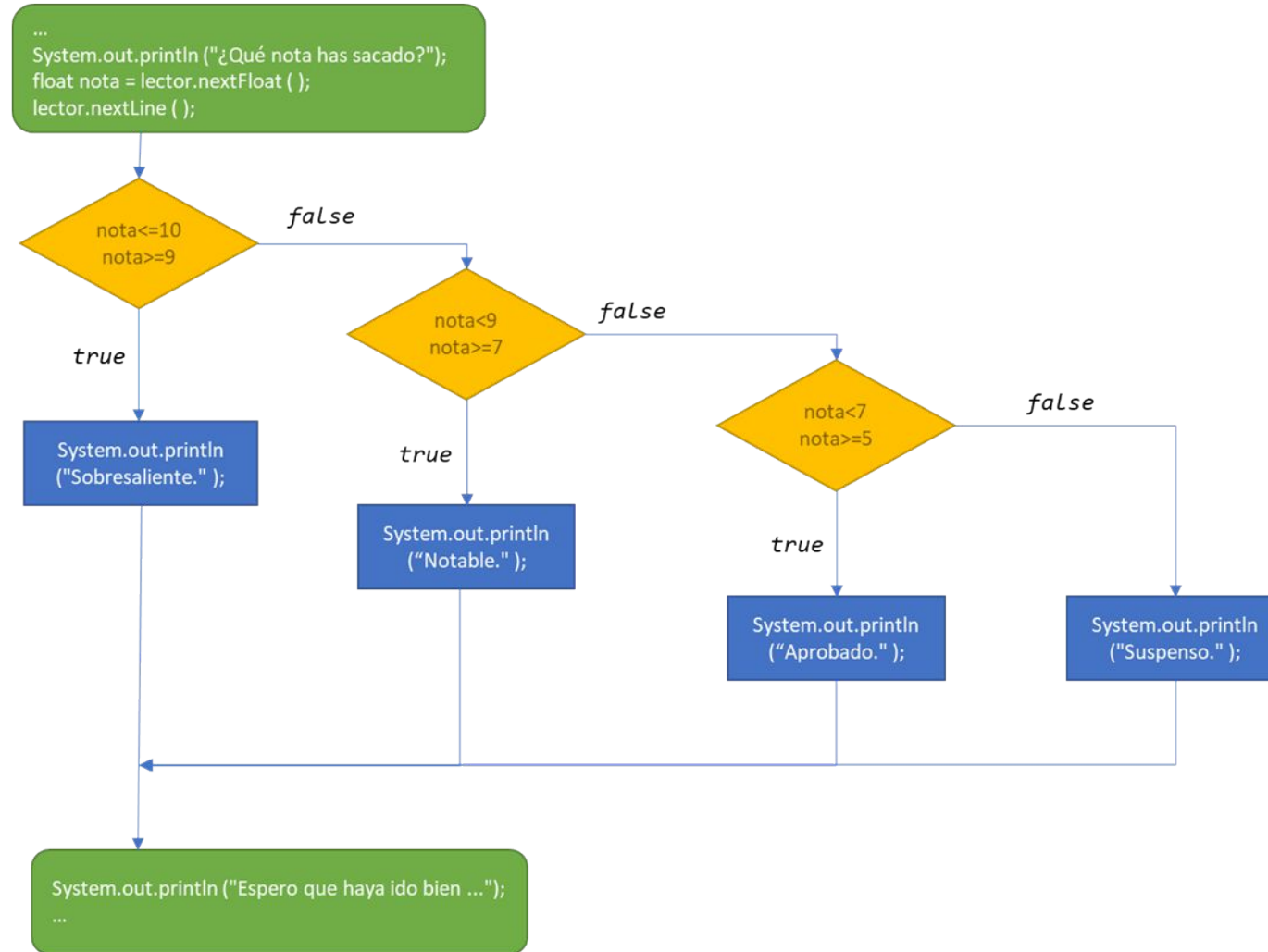
```
... Instrucciones del programa
if (expresión booleana 1) {
    Instrucciones si la 1ª expresión booleana 1 es true;
}
else if (expresión booleana 2) {
    Instrucciones si la expresión booleana 2 es true;
}
else if (expresión booleana 3) {
    Instrucciones si la expresión booleana 3 es true;
}
... se repite la secuencia tantas veces como sea necesario ...

else if (expresión booleana N) {
    Instrucciones si la expresión booleana N es true;
}
else {
    Instrucciones si todas las expresiones booleanas son false;
}
... Resto de instrucciones del programa
```



1.3 - Selección múltiple (Ejemplo)

```
1 import java.util.Scanner;
2 // Un programa que indica la nota en texto a partir de la numérica.
3 public class Nota {
4     public static void main ( String [ ] args ) {
5         Scanner lector = new Scanner ( System.in );
6         // PASO 1 Solicitar la nota
7         System.out.println ("¿Qué nota has sacado?");
8         // PASO 2 Leer la nota
9         float nota = lector.nextFloat ( );
10        lector.nextLine ( );
11        // PASO 3 Mostrar nota en formato texto
12        // Estructura de selección múltiple.
13        // Se entra en el bloque donde la condición lógica evalúe a true.
14        // Si ninguno lo hace, se entra en el bloque else.
15        System.out.print ("Tu nota final es ");
16        if ((nota>=9) && (nota<=10)) {
17            // PASO 3.1
18            System.out.println ("Sobresaliente." );
19        }
20        else if ((nota>=7) && (nota<9)) {
21            // PASO 3.2
22            System.out.println ("Notable.");
23        }
24        else if ((nota>=5) && (nota<7)) {
25            // PASO 3.3
26            System.out.println ("Aprobado.");
27        }
28        else {
29            // PASO 3.4
30            System.out.println ("Suspenso.");
31        }
32        System.out.println ("Espero que haya ido bien ...");
33    }
34 }
```



1.4 - Combinación de estructuras

```
1 import java.util.Scanner ;
2 // Un programa que calcula descuentos.
3 public class Descuento_Modificado {
4     // PASO 1 Se establece el descuento, el precio mínimo y el descuento máximo
5     // Se hace un descuento del 8%.
6     public static final float DESCUENTO = 8;
7     // Se hace descuento por compras de un mínimo de 50 euros.
8     public static final float COMPRA_MIN = 50;
9     // Se hace descuento máximo de 10 euros.
10    public static final float DESC_MAX= 10;
11    public static void main (String [ ] args) {
12        // PASO 2 Se solicita el precio
13        System.out.println ( "¿Cuál es el precio del producto, en euros?" );
14        // PASO 3 Se lee el precio
15        var lector = new Scanner (System.in);
16        float precio = lector.nextFloat ( );
17        lector.nextLine ( );
18        // PASO 4 Comprobar si el precio es positivo.
19        if (precio>0) {
20            // PASO 4.1
21            if ( precio >= COMPRA_MIN ) {
22                // PASO 4.1.1 Calcular descuento
23                float descuent = precio * DESCUENTO / 100;
24                // PASO 4.1.2 Si el descuento supera el máximo, hay que reducirlo.
25                if ( descuent > DESC_MAX ) {
26                    // PASO 4.1.2.1 Reducir descuento
27                    descuent = DESC_MAX ;
28                }
29                // PASO 4.1.3 Aplicar descuento
30                precio = precio - descuent ;
31            }
32            // PASO 4.2 Se muestra el precio final.
33            System.out.println ("El precio final a pagar es de " + precio + " euros.");
34        }
35        else {
36            // PASO 4.3 El precio era negativo. Hay que avisar al usuario.
37            System.out.println ("Precio incorrecto. Es negativo.");
38        }
39    }
40 }
```

Bloque principal

1^{er} nivel

2^º nivel

3^{er} nivel

Estructuras Combinadas

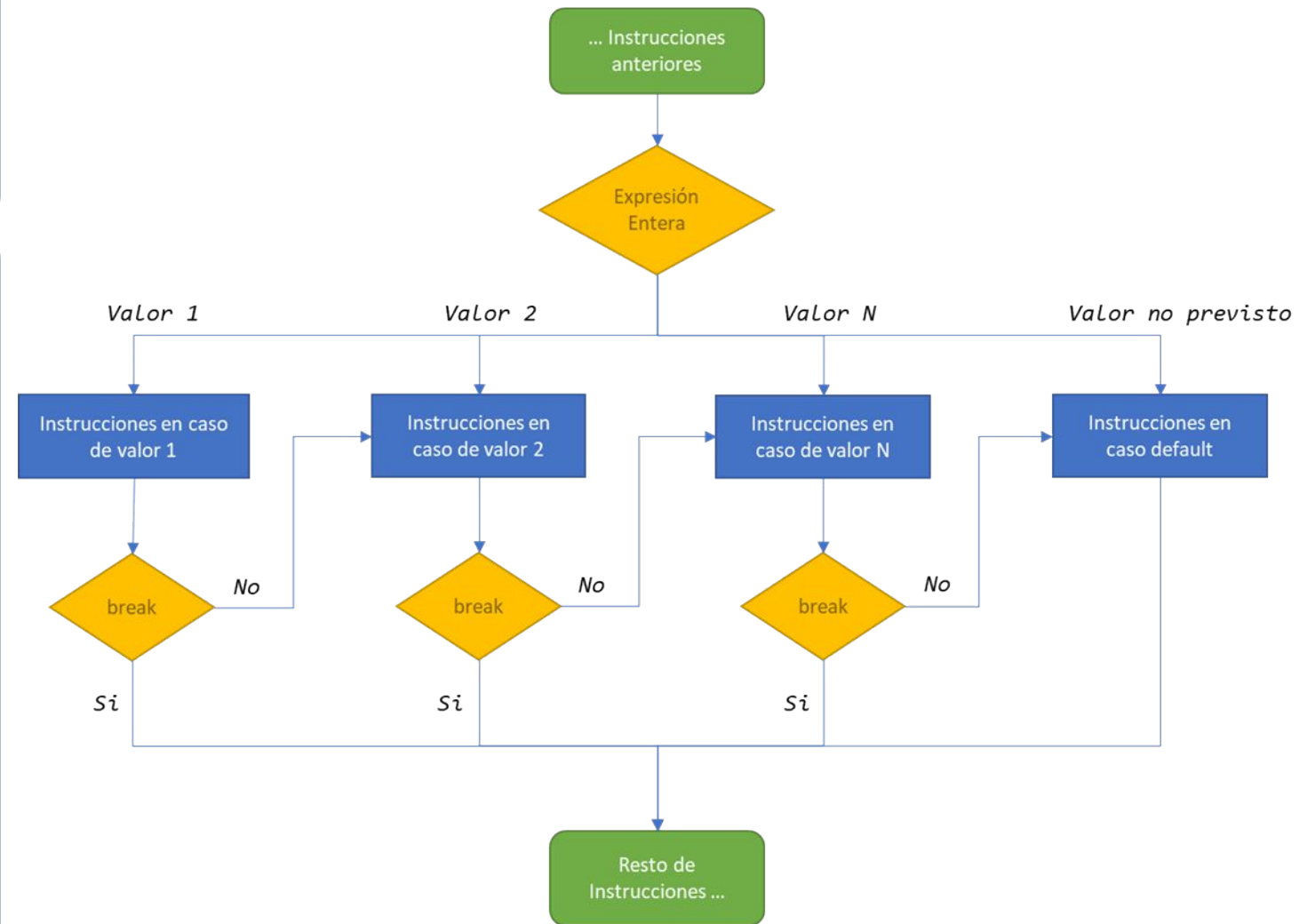
1.5 - Conmutador “switch”

La estructura **Conmutador** permite controlar el hecho de que al cumplirse un caso entre un conjunto finito de casos se ejecute el conjunto de instrucciones correspondiente, siendo la expresión a evaluar de tipo entero o carácter.

... Instrucciones anteriores

```
switch (expresión de tipo entero) {  
  case valor 1:  
    Instrucciones si la expresión evalúa el valor1  
    (Opcionalmente) break;  
  case valor 2:  
    Instrucciones si la expresión evalúa el valor1  
    (Opcionalmente) break;  
  ...  
  case valor N:  
    Instrucciones si la expresión evalúa el valor1  
    (Opcionalmente) break;  
  default:  
    Instrucciones si la expresión no evalúa  
    ninguno de los valores anteriores;  
}
```

Resto de instrucciones ...



1.5 - Conmutador "switch" (Ejemplo)

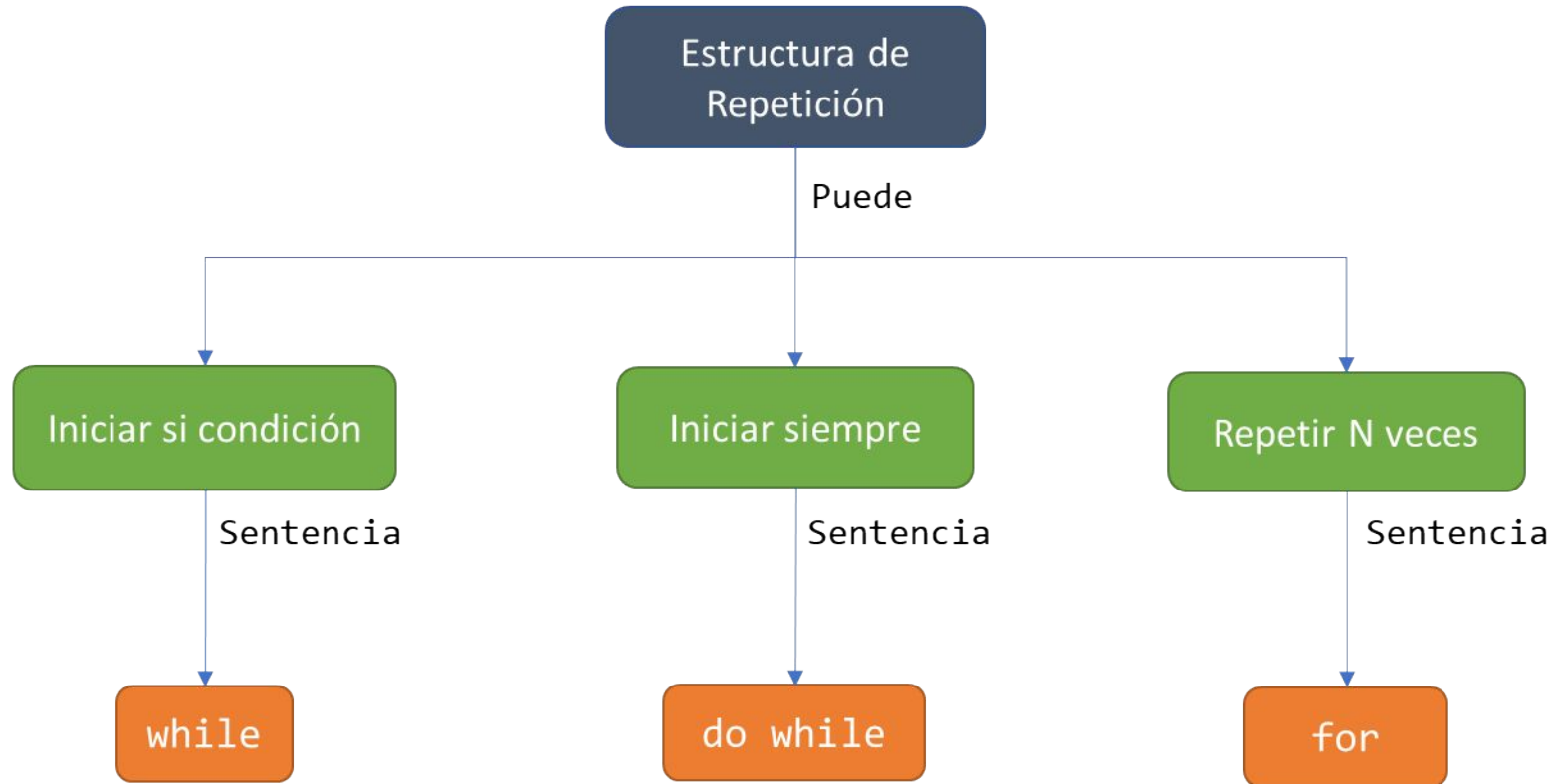
```
1 import java.util.Scanner;
2 // Un programa que da diferentes opciones de cálculo.
3 public class Opciones {
4     public static void main ( String [ ] args ) {
5         Scanner lector = new Scanner ( System.in );
6         // PASO 1 y 2 Solicitar valores y Leerlos
7         System.out.println ("Introduce un valor entero:");
8         int primerValor = Integer.parseInt(lector.next( ));
9         lector.nextLine ( );
10        System.out.println ("Introduce otro:");
11        int segundoValor = Integer.parseInt(lector.next( ));
12        lector.nextLine ( );
13        // PASO 3 y 4 Mostrar menú y leer opción
14        System.out.println ("¿Qué operación quieres hacer?");
15        System.out.println ("[1] Sumar");
16        System.out.println ("[2] Restar");
17        System.out.println ("[3] Multiplicar");
18        System.out.println ("[4] Dividir");
19        System.out.print ("Seleccionar la opción [1-4]:");
20        int opcion = Integer.parseInt(lector.next( ));
21        lector.nextLine ( );
22        int resultado = 0;
23        // PASO 5 Según la opción
24        switch (opcion) {
25            // PASO 5.1 Si es 1, suma dos enteros
26            case 1 :
27                System.out.print ("Has elegido sumar ...");
28                resultado = primerValor + segundoValor;
29                break;
30            // PASO 5.2 Si es 2, resta dos enteros
31            case 2 :
32                System.out.print ("Has elegido restar ...");
33                resultado = primerValor - segundoValor;
34                break;
35            // PASO 5.3 Si es 3, multiplica dos enteros
36            case 3 :
37                System.out.print ("Has elegido multiplicar ...");
38                resultado = primerValor * segundoValor;
39                break;
40            // PASO 5.4 Si es 4, divide dos enteros
41            case 4 :
42                System.out.print ("Has elegido dividir ...");
43                resultado = primerValor / segundoValor;
44                break;
45            // PASO 5.5 Se informa de una selección incorrecta
46            default :
47                System.out.println ("Opción no prevista.");
48        }
49        // PASO 6 Se muestra el resultado
50        System.out.println ("El resultado es " + resultado + " .");
51    }
52 }
```



2 - Estructuras de Repetición

Las **estructuras de repetición** o iterativas permiten repetir una misma secuencia de instrucciones varias veces, mientras se cumpla una cierta condición.

Llamamos **bucle** o ciclo el conjunto de instrucciones que se debe repetir un cierto número de veces, y llamamos **iteración** cada ejecución individual del bucle



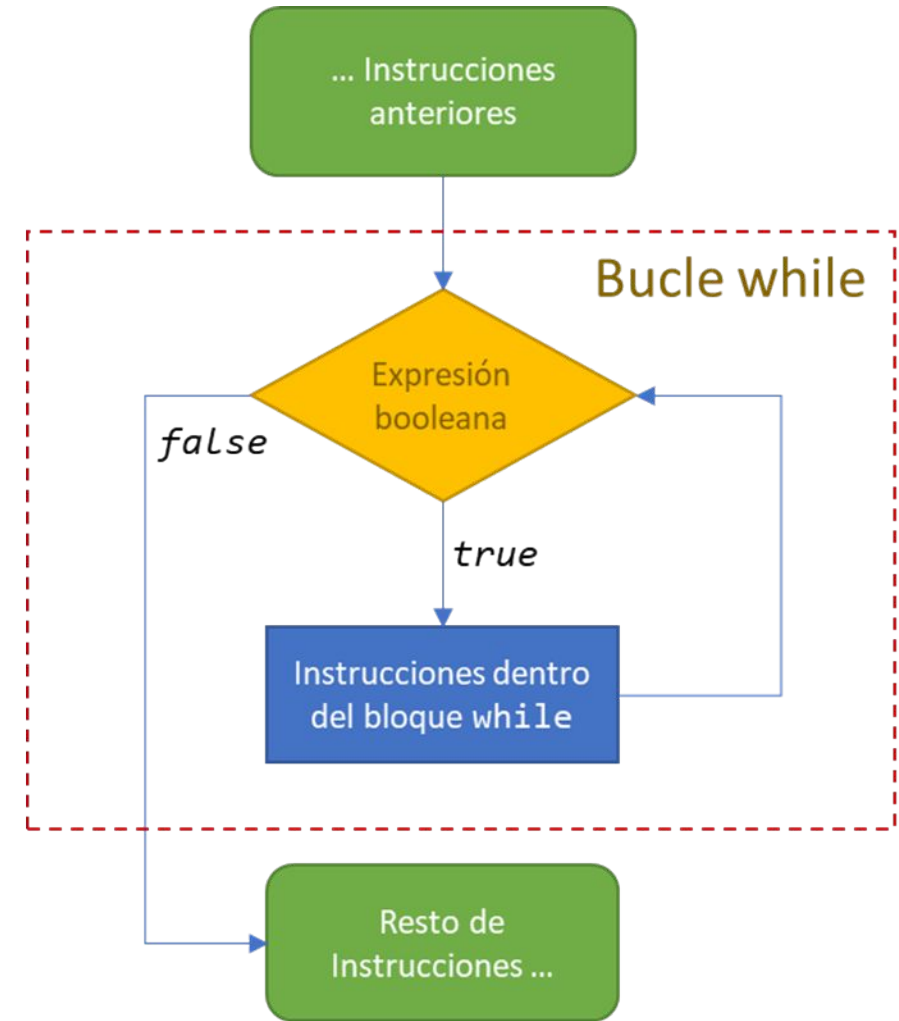
2.1 Bucle “while”

La sentencia **while** permite repetir la ejecución del bucle mientras se verifique la condición lógica. Esta condición se verifica al principio de cada iteración. Si la primera vez, justo cuando se ejecuta la sentencia por primera vez, ya no se cumple, no se ejecuta ninguna iteración.

Instrucciones anteriores ...

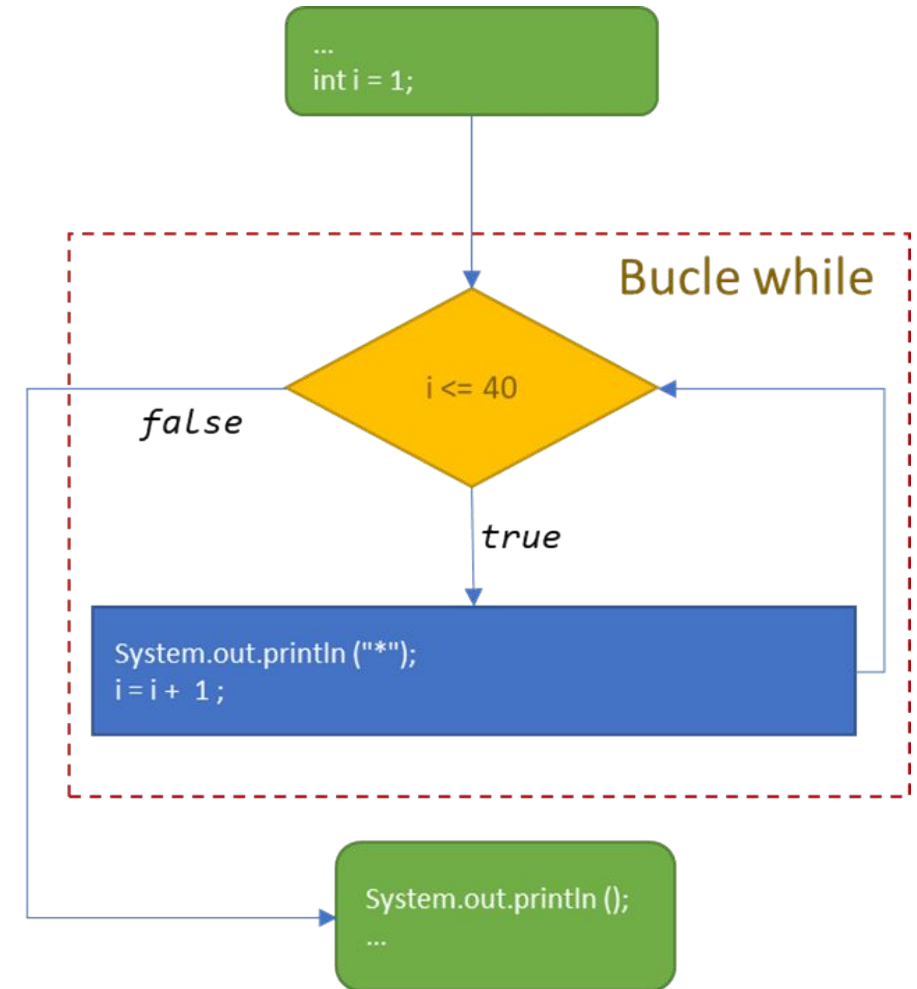
```
while (expresión booleana) {  
    Instrucciones a ejecutar dentro del bucle;  
}
```

Resto de instrucciones ...



2.1 - Bucle “while” (Ejemplo)

```
1 // Un programa que escribe una línea con 40 caracteres '*'.
2 public class Linea {
3     public static void main (String [ ] args) {
4         // inicializar un contador
5         int i = 0 ;
6         //¿Ya hemos escrito * esto 40 veces?
7         while (i<40) {
8             System.out.print ('*') ;
9             // Hemos escrito * una vez, sumamos 1 al contador
10            i = i + 1;
11        }
12        // Forcem un salto de línea
13        System.out.println ( );
14    }
15 }
```



2.1 - Control del Bucle

Un **bucle infinito** es una secuencia de instrucciones dentro de un programa que itera indefinidamente, normalmente porque se espera que se alcance una condición que nunca se llega a producir.

Forzosamente dentro de todo bucle debe haber instrucciones que manipulen el valor de algunas variables que nos permitan controlar la repetición o el final del bucle. Estas variables se conocen como **variables de control**.

Variables de control

Tipos

Contador

Variable de tipo entero que va aumentando o disminuyendo, indicando de manera clara el número de iteraciones que habrá que hacer.

Acumulador

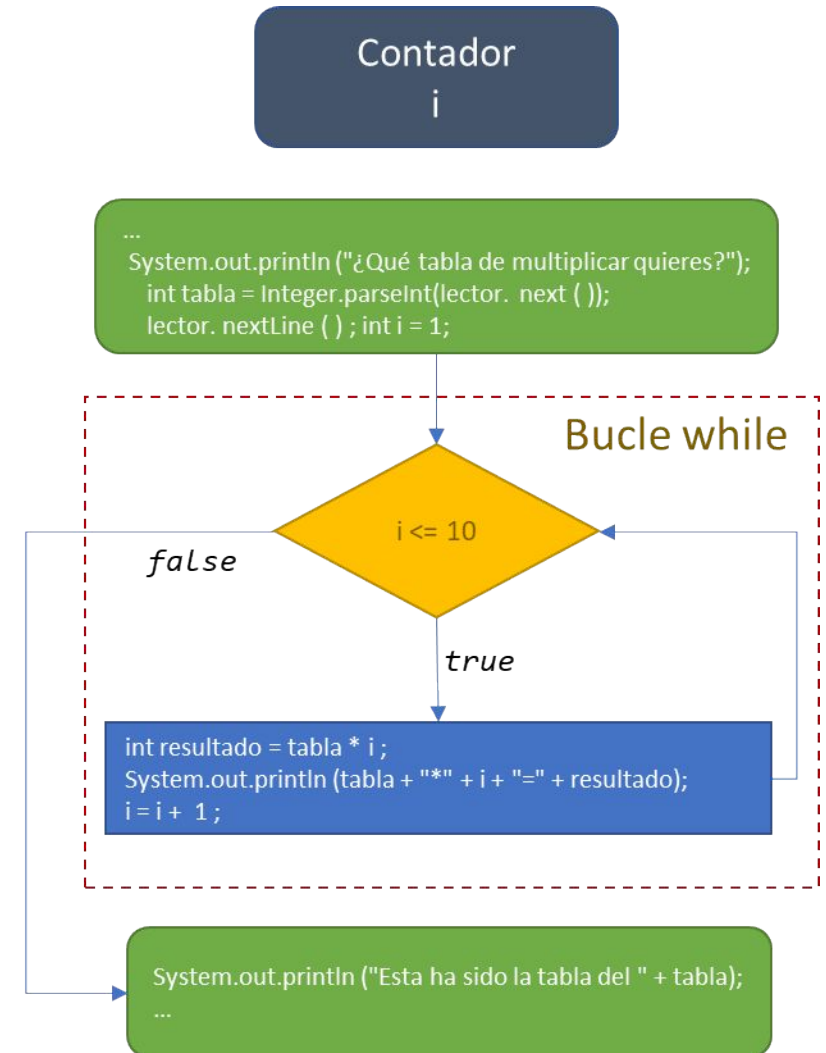
Variable en la que se van acumulando directamente los cálculos que se quieren hacer, de manera que al alcanzar cierto valor se considera que ya no es necesario hacer más iteraciones.

Semáforo

Variable que sirve como interruptor explícito de si hay que seguir haciendo iteraciones. Cuando ya no queremos hacer más, el código simplemente se encarga de asignarle el valor específico que servirá para que la condición evalúe false.

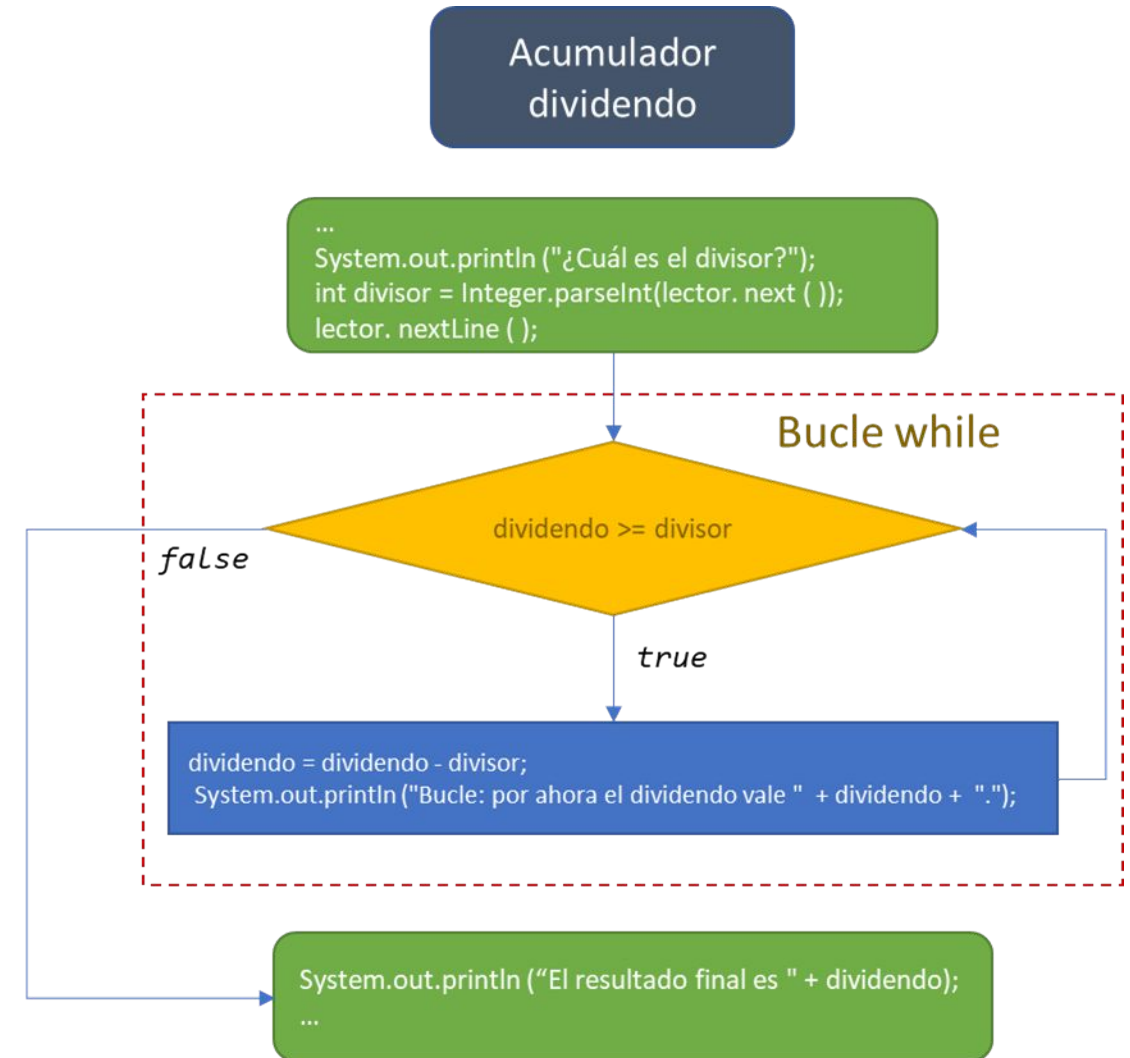
2.1 - Bucle "while" (Contador)

```
1 import java.util.Scanner ;
2 // Un programa que muestra la tabla de multiplicar de un número.
3 public class Multiplicar {
4     public static void main ( String [ ] args ) {
5         // inicializa la biblioteca.
6         Scanner lector = new Scanner (System.in);
7         // Pregunta el número.
8         System.out.println ("¿Qué tabla de multiplicar quieres?");
9         int tabla = Integer.parseInt(lector. next ( ));
10        lector. nextLine ( ) ;
11        // El contador servirá para hacer cálculos.
12        int i = 1;
13        while (i <= 10) {
14            int resultado = tabla * i ;
15            System.out.println (tabla + "*" + i + "=" + resultado);
16            i = i + 1 ;
17        }
18        System . out . println ("Esta ha sido la tabla del " + tabla);
19    }
20 }
```



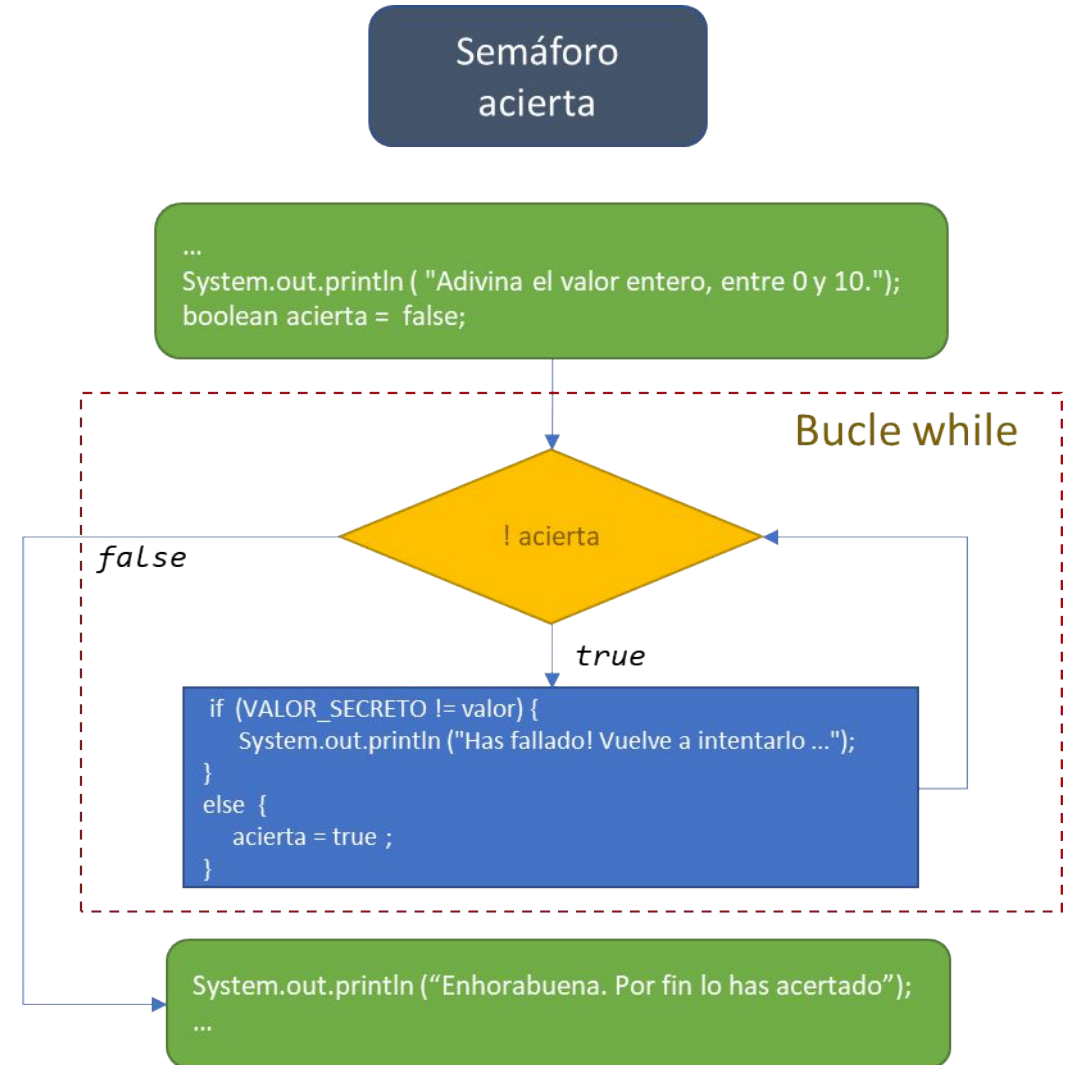
2.1 - Bucle “while” (Acumulador)

```
1 import java.util.Scanner ;
2 // Un programa que lee un entero y lo muestra por pantalla.
3 public class Modulo {
4     public static void main ( String [ ] args ) {
5         Scanner lector = new Scanner (System.in);
6         // PASO 1 Solicitar dividendo
7         System.out.println ( "¿Cuál es el dividendo?" );
8         // PASO 2 Leer dividendo
9         int dividendo = Integer.parseInt(lector.next ( ));
10        lector.nextLine ( );
11        // PASO 3 Solicitar divisor
12        System.out.println ( "¿Cuál es el divisor?" );
13        // PASO 4 Leer divisor
14        int divisor = Integer.parseInt(lector.next ( ));
15        lector.nextLine ( );
16        // PASO 5 ¿El dividendo es menor que el divisor?
17        while (dividendo >= divisor) {
18            // PASO 6 Restar el divisor al dividendo
19            dividendo = dividendo - divisor;
20            System.out.println ( "Bucle: por ahora el dividendo vale " + dividendo + "." );
21            // PASO 7 repetimos el bucle para evaluar la condición
22        }
23        System.out.println ( "El resultado final es " + dividendo + "." );
24    }
25 }
```



2.1 - Bucle “while” (Acumulador)

```
1 import java.util.Scanner ;
2 // Un programa en el que hay que adivinar un número.
3 public class Adivina_2 {
4     // PASO 1 Decidir número
5     public static final int VALOR_SECRETO = 4;
6     public static void main ( String [ ] args ) {
7         Scanner lector = new Scanner (System .in);
8         System.out.println ( "Empezamos el juego." );
9         System.out.println ( "Adivina el valor entero, entre 0 y 10." );
10        boolean acierta = false;
11        while ( ! acierta ) {
12            // PASO 2 Solicitar número
13            System.out.println ( "¿Qué valor crees que es?" ) ;
14            // PASO 3 Leer número
15            int valor =Integer.parseInt(lector. next ( ) );
16            lector.nextLine ( ) ;
17            // PASO 4 ¿El número es correcto?
18            if (VALOR_SECRETO != valor) {
19                // PASO 4.1 Avisar del fallo
20                System.out.println ( "Has fallado! Vuelve a intentarlo ..." );
21            }
22            else {
23                acierta = true ;
24            }
25        }
26        // PASO 5 Mostrar felicitación
27        System.out.println ( "Enhorabuena. Por fin lo has acertado!" ) ;
28    }
29 }
```



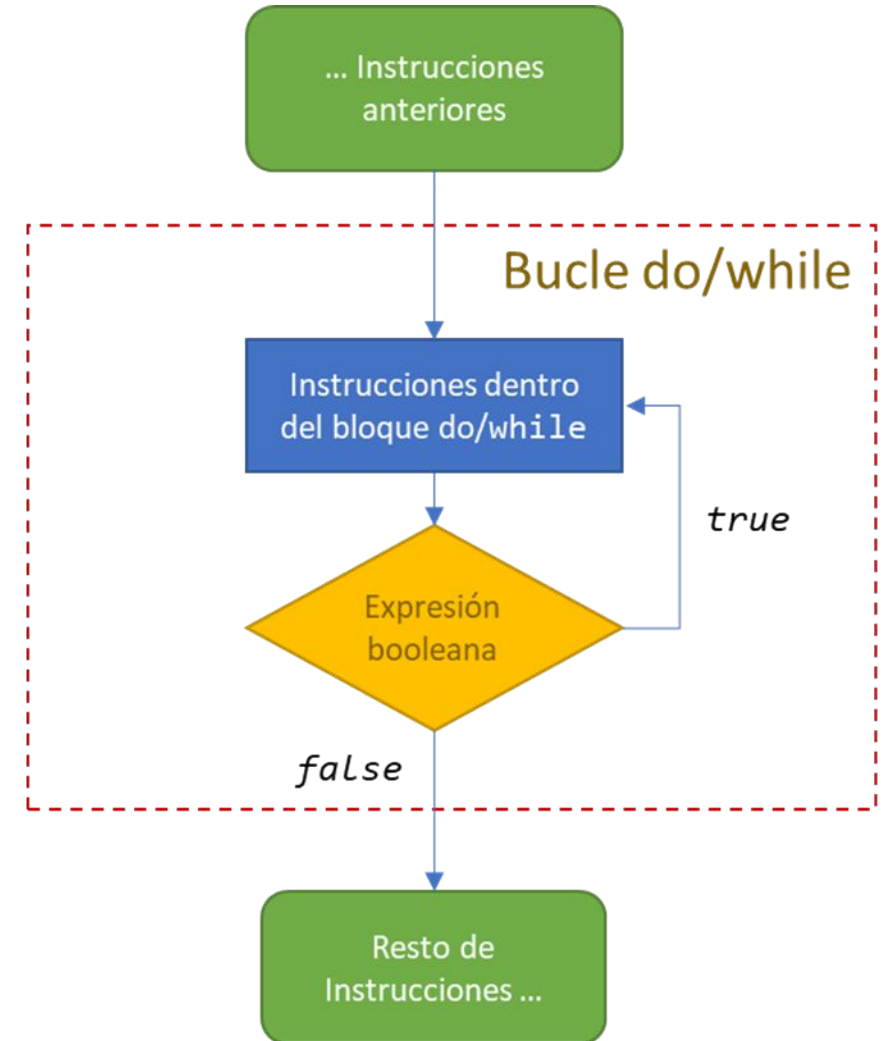
2.2 - Bucle “do – while”

La sentencia **do / while** permite repetir la ejecución del bucle mientras se verifique la condición lógica. A diferencia de la sentencia **while**, la condición se verifica al final de cada iteración. Por lo tanto, independientemente de cómo evalúe la condición, al menos siempre se llevará a cabo la primera iteración.

Instrucciones anteriores ...

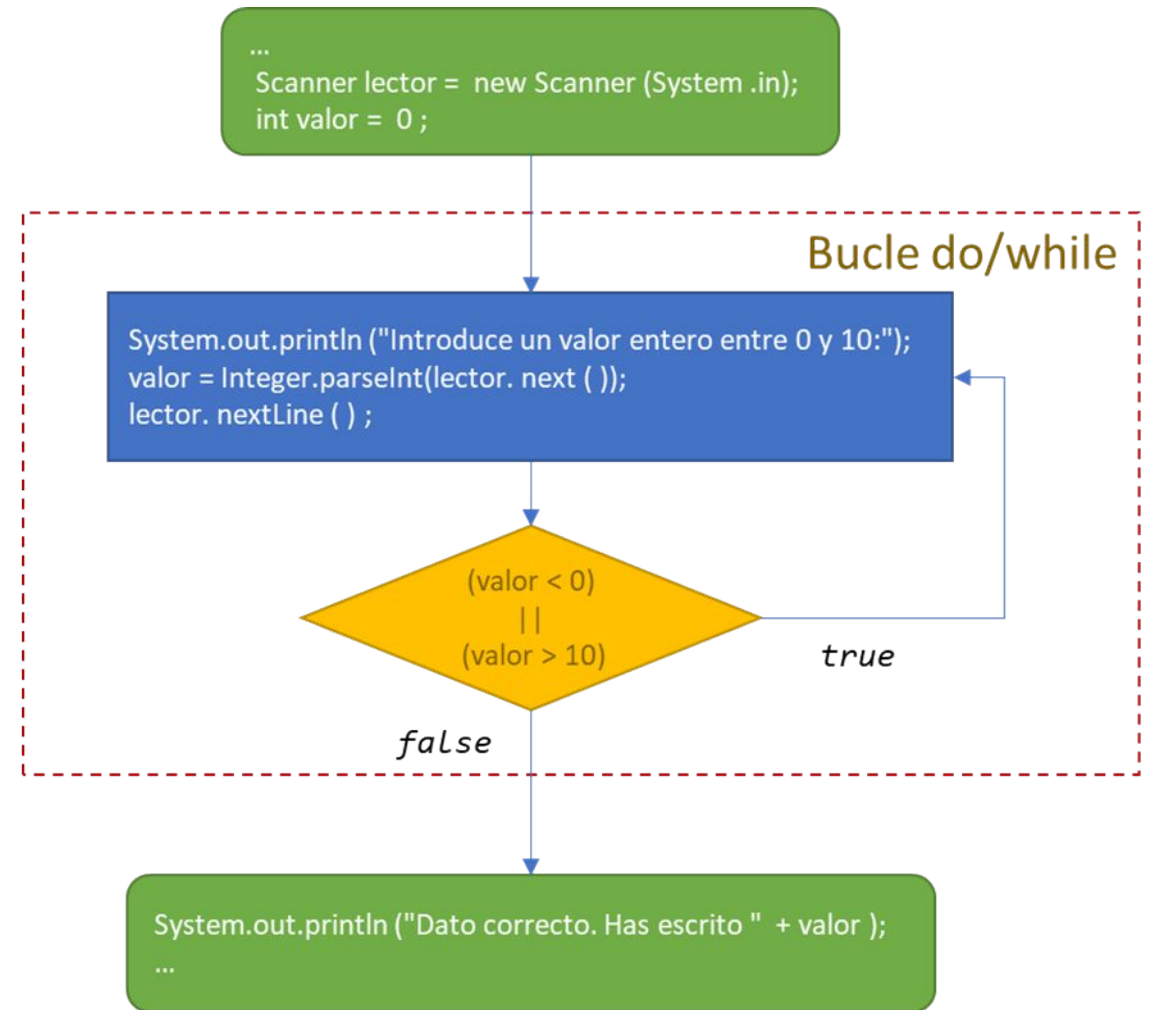
```
do {  
    Instrucciones a ejecutar dentro del bucle;  
} while (expresión booleana)
```

Resto de instrucciones ...



2.2 - Bucle “do – while” (Ejemplo)

```
1 import java.util.Scanner ;
2 // Vamos a leer un entero entre 0 y 10.
3 public class Comprobar_dato {
4     public static void main ( String [ ] args ) {
5         Scanner lector = new Scanner (System .in);
6         int valor = 0 ;
7         do {
8             // PASO 1 Solicitar dato, como mínimo se pregunta una vez.
9             System.out.println ("Introduce un valor entero entre 0 y 10:");
10            // PASO 2 Leer dato
11            valor = Integer.parseInt(lector. next ( ));
12            lector. nextLine ( ) ;
13            // PASO 3 Comprobar dato.
14        } while ((valor < 0) || (valor > 10));
15        // PASO 4: Todo correcto
16        System.out.println ("Dato correcto. Has escrito " + valor );
17    }
18 }
```



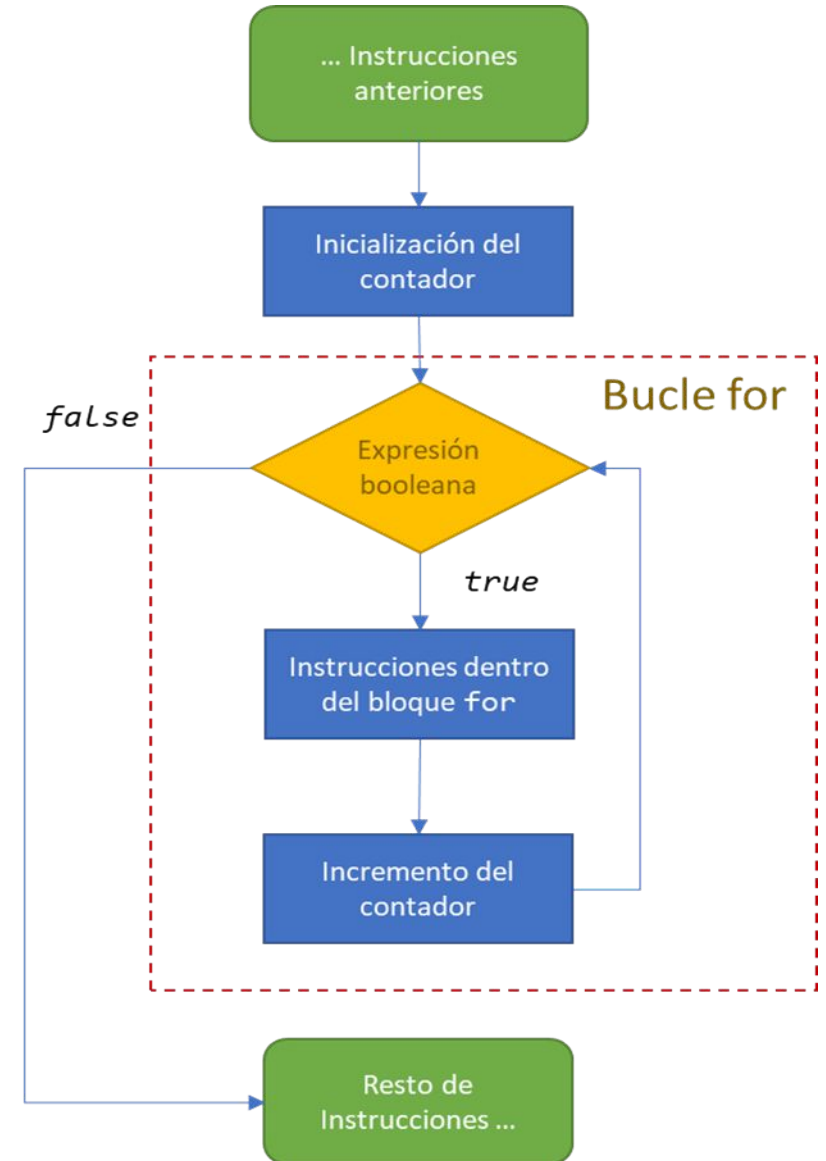
2.3 - Bucle "for"

La sentencia **for** permite repetir un número determinado de veces un conjunto de instrucciones.

Instrucciones anteriores ...

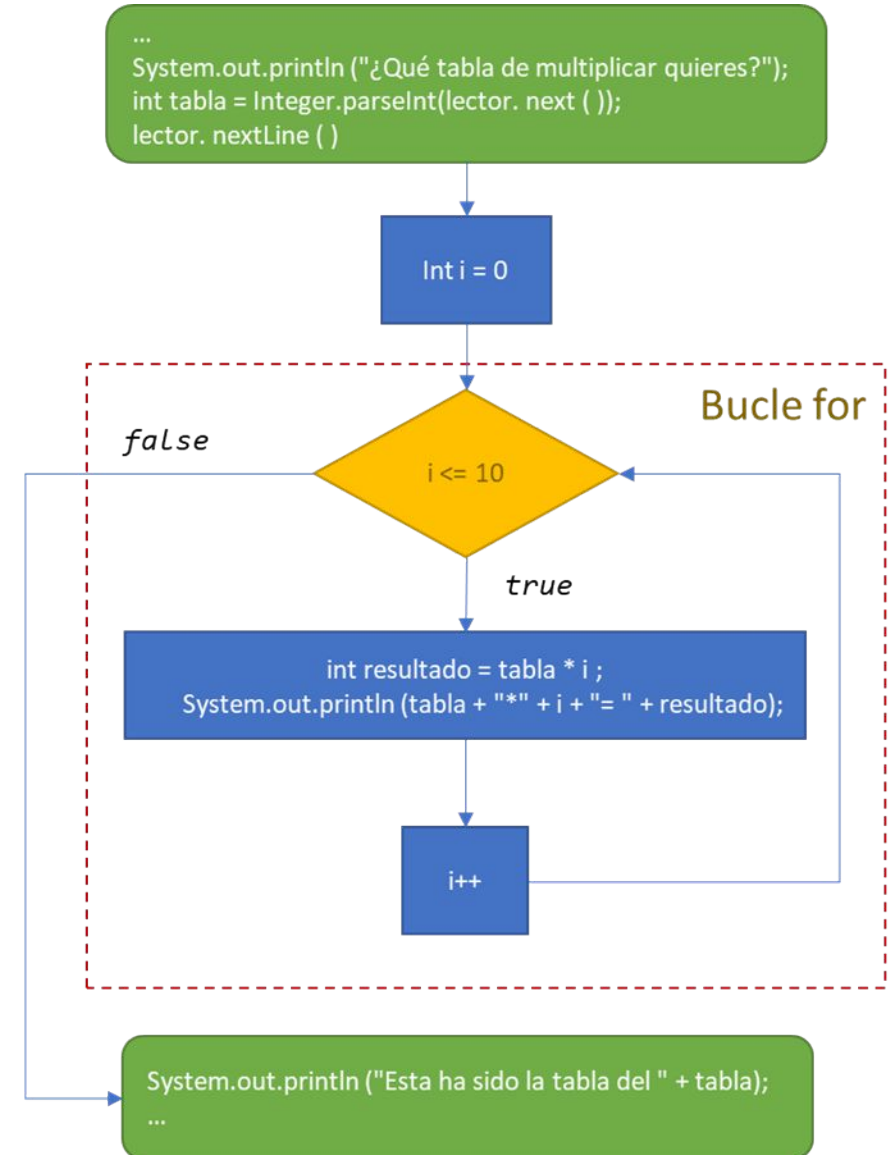
```
for (Inicialización contador; Expresión booleana; Incremento contador) {  
    Instrucciones a ejecutar dentro del bucle;  
}
```

Resto de instrucciones ...



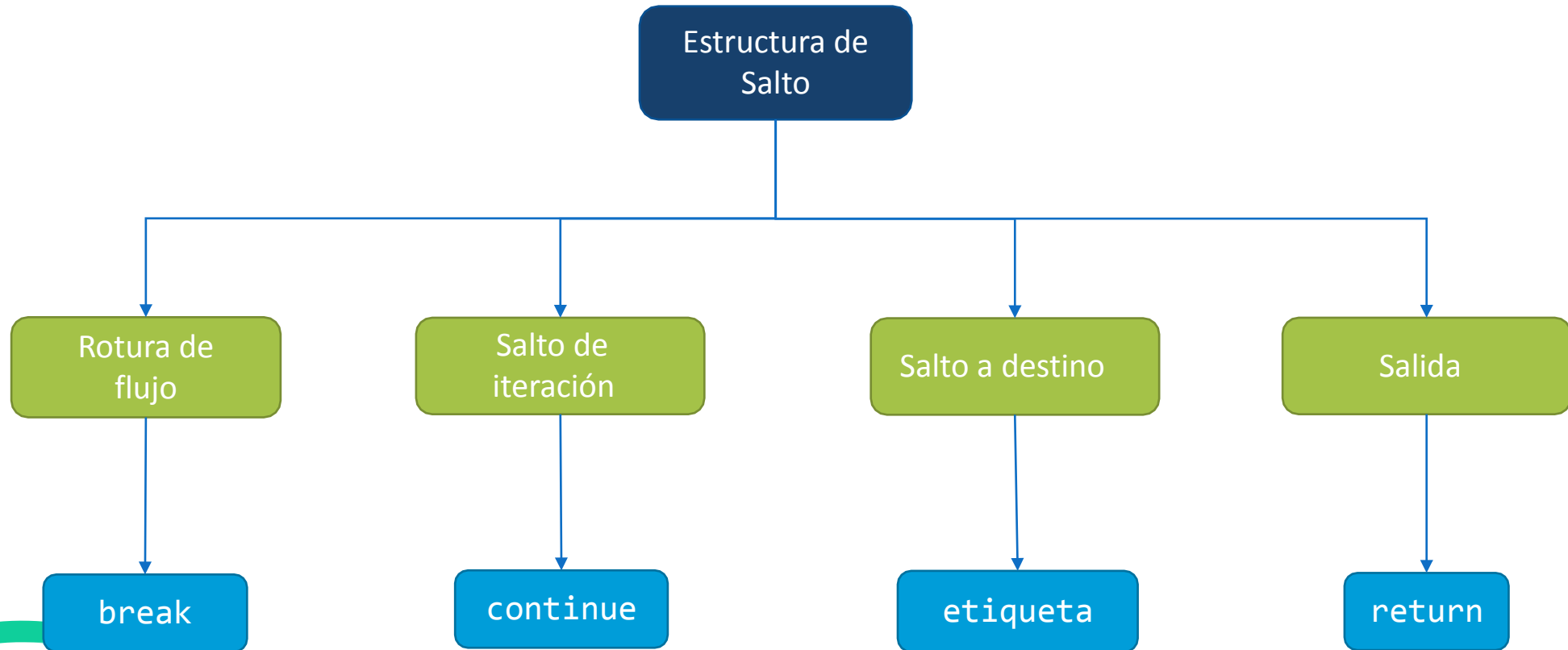
2.3 - Bucle "for" (Ejemplo)

```
1 import java.util.Scanner ;
2 // Un programa que muestra la tabla de multiplicar de un número usando for.
3 public class Multiplicar_2 {
4     public static void main ( String [ ] args ) {
5         // inicializa la biblioteca.
6         Scanner lector = new Scanner (System.in);
7         // Pregunta el número.
8         System.out.println ("¿Qué tabla de multiplicar quieres?");
9         int tabla = Integer.parseInt(lector. next ( ));
10        lector.nextLine ( ) ;
11        // El contador servirá para hacer cálculos.
12        // En lugar de i++ también e puede escribit i=i+1
13        for (int i = 0 ; i <= 10 ; i++) {
14            int resultado = tabla * i ;
15            System.out.println (tabla + "*" + i + "= " + resultado);
16        }
17        System.out.println ("Esta ha sido la tabla del " + tabla);
18    }
19 }
```



3 - Estructuras de salto

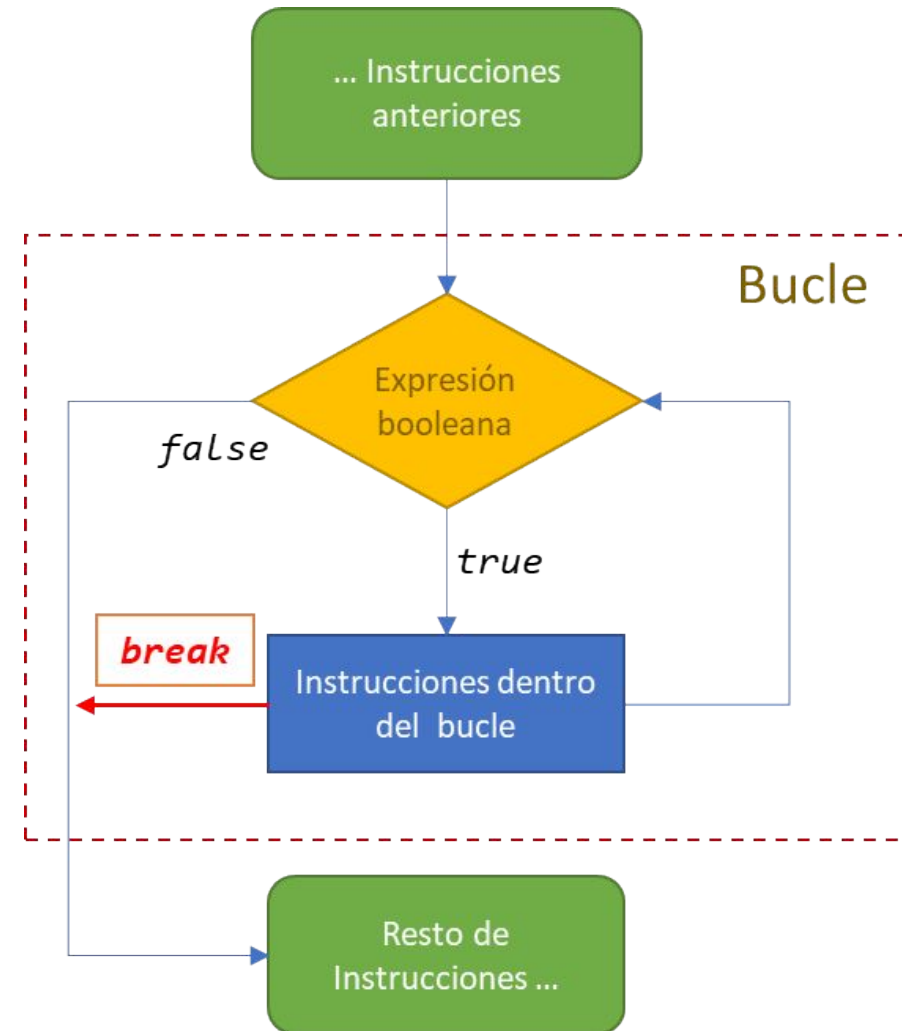
Una sentencia de **salto** nos permite transferir el control del programa de forma incondicional.



3.1 - Sentencia "break"

La sentencia **break** sirve para romper el flujo de control de un bucle, aunque no se haya cumplido la condición del bucle.

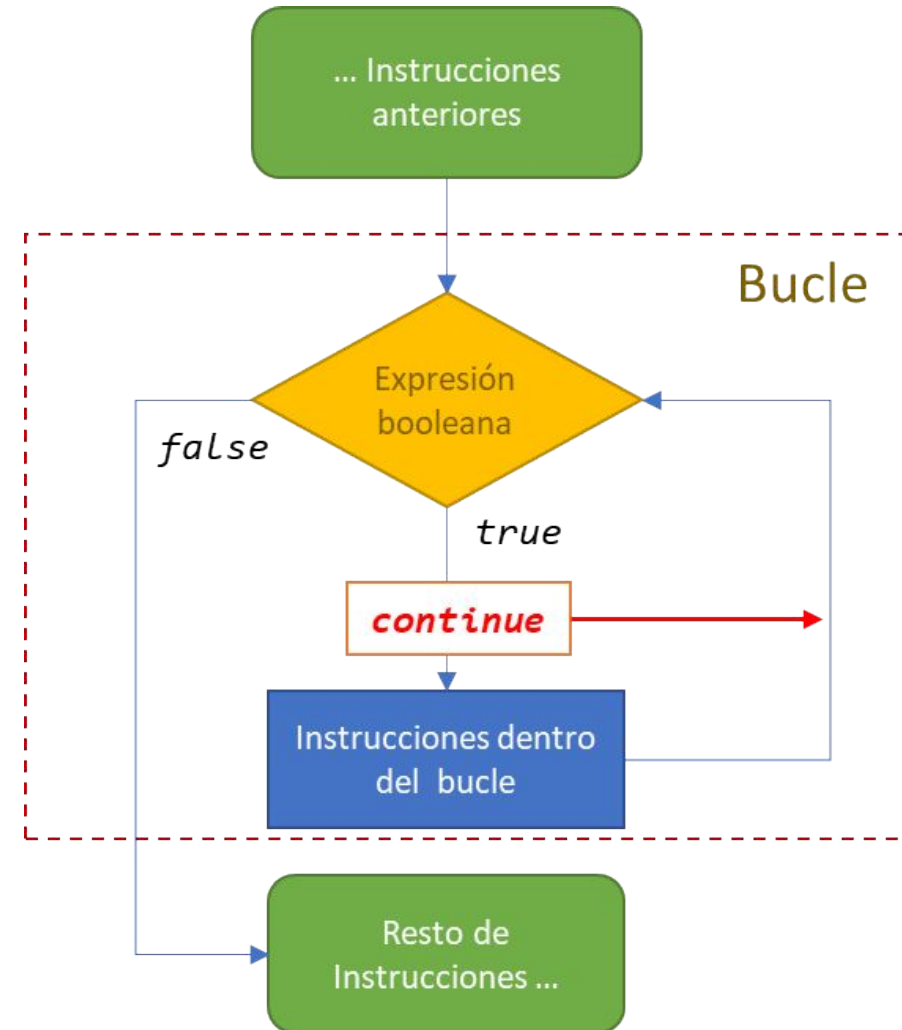
```
1 // Un programa que muestra el uso de la sentencia break.
2 public class Uso_break {
3     // Declaramos el punto de salida del bucle.
4     public static final int FIN = 10;
5     public static void main ( String [ ] args ) {
6         System.out.println ("Este programa imprime los números del 1 al " + FIN);
7         //iniciamos el bucle.
8         for (int i = 0 ; i <= 100 ; i++) {
9             if (i==FIN) {
10                // Cuando se alcanza el valor del FINALIZADOR se sale del bucle
11                System.out.println ("Valor final: " + i );
12                break;
13            }
14            System.out.println (i);
15        }
16        System.out.println ("Fin del programa");
17    }
18 }
```



3.2 - Sentencia "continue"

La sentencia **continue** forzará a que se ejecute la siguiente iteración del bucle, sin tener en cuenta las instrucciones que pudiera haber después del continue, y hasta el final del código del bucle.

```
1 // Un programa que muestra el uso de la sentencia continue.
2 public class Uso_continue {
3     // Declaramos el punto de salida del bucle.
4     public static final int FIN = 10;
5     public static void main ( String [ ] args ) {
6         System.out.println ("Este programa imprime los números del 1 al 100 ...");
7         //iniciamos el bucle.
8         //imprimimos los números pares del 1 al 100
9         for (int i = 0 ; i <= 100 ; i++) {
10            //si el numero no es par salta otra iteración
11            if (i%2 != 0) continue;
12            // Cuando se alcanza el valor del FINALIZADOR se sale del bucle
13            System.out.println (i);
14        }
15        System.out.println ("Fin del programa");
16    }
17 }
```



3.4 – Sentencia “return”

Un **return** es la instrucción utilizada en las funciones para poder devolver un valor al procedimiento o función que la ha invocado.

```
1 // Un programa que muestra el uso de la sentencia return.
2 import java.util.Scanner ;
3 public class Uso_return {
4
5     public static int suma(int numero1, int numero2) {
6         int resultado;
7         resultado = numero1 + numero2;
8         //Mediante return, este método devuelve el resultado de la suma
9         return resultado;
10    }
11    public static void main(String[] args) {
12        Scanner lector = new Scanner (System.in);
13        //Estas variables almacenarán los operandos
14        // Solicitamos que el usuario introduzca dos números por consola
15        System.out.println ("Introduce el primer operando:");
16        int primer_numero = Integer.parseInt(lector. next ( ));
17        lector. nextLine ( );
18        System.out.println ("Introduce el segundo operando: ");
19        int segundo_numero = Integer.parseInt(lector. next ( ));
20        lector. nextLine ( );
21        //Imprimimos los números introducidos
22        System.out.println ("Los operandos son: " + primer_numero + " y " + segundo_numero);
23        System.out.println ("obteniendo su suma... ");
24        //Invocamos al método que realiza la suma, pasándole los parámetros adecuados
25        System.out.println ("La suma de ambos operandos es: " + suma(primer_numero,segundo_numero));
26    }
27 }
```